

I. CROSS JOIN

Schema for Q1–Q2

Table: Color

ColorID	Name
1	Red
2	Blue
3	Green

Table: Size

SizeID	Label
1	S
2	M
3	L

Q1. List all possible combinations of colors and sizes.

```
SELECT c.Name AS Color, s.Label AS Size
FROM Color c
CROSS JOIN Size s;
```

Step-wise explanation:

1. Take every row from `Color` (3 rows).
2. For each color row, pair it with **every** row from `Size` (3 rows).
3. Result = $3 \times 3 = 9$ rows: (Red,S), (Red,M), (Red,L), (Blue,S), ..., (Green,L).

Q2. Count how many combinations exist in total.

```
SELECT COUNT(*) AS TotalCombinations
FROM Color c
CROSS JOIN Size s;
```

Steps:

1. Same Cartesian product as Q1 (9 rows).
2. COUNT(*) counts rows in that product.
3. Output: TotalCombinations = 9.

Schema for Q3–Q4

Table: Meal

+-----+-----+	
MealID MealName	
+-----+-----+	
1 Idli	
2 Dosa	

Table: Drink

+-----+-----+	
DrinkID DrinkName	
+-----+-----+	
10 Coffee	
11 Tea	
12 Lemon Juice	

Q3. Generate a menu of all meal–drink combos.

```
SELECT m.MealName, d.DrinkName
FROM Meal m
CROSS JOIN Drink d
ORDER BY m.MealName, d.DrinkName;
```

Steps:

1. CROSS JOIN makes 2 (meals) × 3 (drinks) = 6 rows.
2. Each meal appears once with each drink.
3. ORDER BY sorts first by meal, then drink.

Q4. Show only those combinations where the drink is not Tea, but still use CROSS JOIN.

```
SELECT m.MealName, d.DrinkName
FROM Meal m
CROSS JOIN Drink d
WHERE d.DrinkName <> 'Tea';
```

Steps:

1. CROSS JOIN first generates all 6 pairs.
2. WHERE d.DrinkName <> 'Tea' removes the 2 pairs with Tea.
3. Final result: 4 rows (each meal with Coffee and Lemon Juice).

Schema for Q5–Q6

Table: Subject

SubID	SubName
1	DBMS
2	Web Tech

Table: DaySlot

SlotID	Day
1	Monday
2	Tuesday
3	Friday

Q5. Create a timetable skeleton for all subject–day combinations.

```
SELECT s.SubName, d.Day
FROM Subject s
CROSS JOIN DaySlot d
ORDER BY d.Day, s.SubName;
```

Steps:

1. CROSS JOIN gives $2 \times 3 = 6$ (subject, day) rows.
2. No condition; all pairs appear.
3. ORDER BY just arranges them by day then subject.

Q6. Suppose you only conduct classes on Monday and Friday. Show possible class slots using CROSS JOIN plus filter.

```
SELECT s.SubName, d.Day
FROM Subject s
CROSS JOIN DaySlot d
WHERE d.Day IN ('Monday', 'Friday');
```

Steps:

1. Start with all 6 pairs from the CROSS JOIN.
2. WHERE d.Day IN ('Monday','Friday') keeps only 4 rows.
3. You get: DBMS/Web Tech on Monday and Friday.

II. EQUI JOIN

Equi join = ... ON table1.col = table2.col.

Schema for Q1–Q2

Table: Employee

EmpID	EmpName	DeptID
1	Asha	10
2	Ravi	20
3	Meera	10
4	John	30

Table: Department

DeptID	DeptName
10	HR
20	Finance
40	R&D

Q1. List employees with their department names.

```
SELECT e.EmpName, d.DeptName
FROM Employee e
JOIN Department d
ON e.DeptID = d.DeptID;
```

Steps:

1. For each row in Employee, find a row in Department where DeptID matches.
2. Matches:
 - Emp 1 (DeptID 10) ↔ HR
 - Emp 2 (20) ↔ Finance
 - Emp 3 (10) ↔ HR
 - Emp 4 (30) → no matching department, so excluded.
3. Result = 3 rows: Asha–HR, Ravi–Finance, Meera–HR.

Q2. Show department-wise employee count (only for departments that have employees).

```
SELECT d.DeptName, COUNT(*) AS EmpCount
FROM Employee e
JOIN Department d
ON e.DeptID = d.DeptID
GROUP BY d.DeptName;
```

Steps:

1. Same join as Q1, giving 3 joined rows.
2. Group by department name: HR (2 employees), Finance (1).
3. Output: (HR, 2), (Finance, 1). R&D (no employees) not shown.

Schema for Q3–Q4

Table: Student

RollNo	Name	ClassID
101	Ankit	C1
102	Bhavna	C2
103	Chirag	C1

Table: Class

ClassID	ClassName
---------	-----------

+-----+-----+	
C1	BTech CS
C2	BTech ECE
C3	BTech ME

Q3. Display student name, roll number, and class name.

```
SELECT s.RollNo, s.Name, c.ClassName
FROM Student s
INNER JOIN Class c
ON s.ClassID = c.ClassID;
```

Steps:

1. Join where Student.ClassID = Class.ClassID.
2. Matches: 101–C1, 102–C2, 103–C1; C3 unused.
3. Output 3 rows with full info.

Q4. Find all classes that currently have at least one student enrolled (distinct).

```
SELECT DISTINCT c.ClassID, c.ClassName
FROM Student s
JOIN Class c
ON s.ClassID = c.ClassID;
```

Steps:

1. Same inner join as Q3 (3 rows).
2. Use DISTINCT to remove repeated classes (C1 appears twice).
3. Final result: C1 (BTech CS), C2 (BTech ECE).

Schema for Q5–Q6

Table: OrderHeader

+-----+-----+-----+		
OrderID	CustID	OrderDate
+-----+-----+-----+		
1	201	2025-02-01
2	202	2025-02-02

| 3 | 201 | 2025-02-03 |

Table: Customer

CustID	CustName
201	Kumar
202	Latha
203	Deepak

Q5. Show order ID, date, and customer name.

```
SELECT o.OrderID, o.OrderDate, c.CustName
FROM OrderHeader o
JOIN Customer c
ON o.CustID = c.CustID;
```

Steps:

1. For each order, find customer by CustID.
2. Orders 1 & 3 → Kumar; Order 2 → Latha; Cust 203 unused.
3. Output 3 joined rows.

Q6. Show customers who have never placed any order.

```
SELECT c.CustID, c.CustName
FROM Customer c
LEFT JOIN OrderHeader o
ON c.CustID = o.CustID
WHERE o.OrderID IS NULL;
```

(Uses equi join inside a LEFT JOIN.)

Steps:

1. LEFT JOIN: keep all customers; match any orders on CustID.
2. For 201 and 202, there is at least one matching order.
3. For 203, no matching order → o.OrderID is NULL.
4. WHERE o.OrderID IS NULL filters to only such rows → Deepak.

III. NON-EQUI JOIN

Non-equi joins use <, >, BETWEEN, etc.

Schema for Q1–Q2

Table: Employee

EmpID	EmpName	Salary
1	Asha	18000
2	Ravi	26000
3	Meera	42000
4	John	52000

Table: SalaryGrade

Grade	MinSal	MaxSal
G1	0	20000
G2	20001	40000
G3	40001	60000

Q1. Assign each employee a salary grade based on range.

```
SELECT e.EmpName, e.Salary, g.Grade
FROM Employee e
JOIN SalaryGrade g
ON e.Salary BETWEEN g.MinSal AND g.MaxSal;
```

Steps:

- For each employee, compare their salary with every grade row:
 - Asha 18000 → falls in 0–20000 → G1.
 - Ravi 26000 → 20001–40000 → G2.
 - Meera 42000 → 40001–60000 → G3.
 - John 52000 → 40001–60000 → G3.
- BETWEEN condition matches exactly one grade per employee.
- Output 4 rows with appropriate grade.

Q2. List employees who are above the minimum of the highest grade.

```
SELECT e.EmpName, e.Salary
FROM Employee e
JOIN SalaryGrade g
ON e.Salary > g.MinSal
WHERE g.Grade = 'G3';
```

Steps:

1. First, for grade G3, MinSal = 40001.
2. Join keeps employees whose salary > 40001.
3. Meera(42000) and John(52000) satisfy; Asha & Ravi do not.
4. WHERE g.Grade='G3' ensures we only use min of grade G3.

Schema for Q3–Q4

Table: StudentMarks

RollNo	Marks
1	92
2	76
3	61
4	48

Table: GradeRules

Grade	MinM	MaxM
A	85	100
B	70	84
C	50	69
D	0	49

Q3. Assign letter grade to each student.

```
SELECT s.RollNo, s.Marks, g.Grade
FROM StudentMarks s
JOIN GradeRules g
ON s.Marks BETWEEN g.MinM AND g.MaxM;
```

Steps:

1. For each mark, find grade rule whose range covers it:
 - 92 → A; 76 → B; 61 → C; 48 → D.

2. BETWEEN ensures exactly one matching grade per student.
3. Output 4 rows.

Q4. Find all students who are just one grade below 'A' (i.e., in grade B).

```
SELECT s.RollNo, s.Marks
FROM StudentMarks s
JOIN GradeRules g
ON s.Marks BETWEEN g.MinM AND g.MaxM
WHERE g.Grade = 'B';
```

Steps:

1. Non-equi join assigns grades as in Q3.
2. WHERE g.Grade='B' filters to marks in 70–84.
3. Result: RollNo 2 (76).

Schema for Q5–Q6

Table: Product

ProdID	ProdName	Price
1	Pen	10
2	Notebook	55
3	Bag	850

Table: DiscountSlab

SlabID	MinPrice	MaxPrice	DiscPct
1	0	50	5
2	51	500	10
3	501	1000	15

Q5. For each product, show applicable discount percentage.

```
SELECT p.ProdName, p.Price, d.DiscPct
FROM Product p
JOIN DiscountSlab d
ON p.Price BETWEEN d.MinPrice AND d.MaxPrice;
```

Steps:

1. Compare price with each slab's range:
 - Pen 10 → 0–50 → 5%
 - Notebook 55 → 51–500 → 10%
 - Bag 850 → 501–1000 → 15%
2. Each product joins with exactly one discount row.
3. Output prices with discounts.

Q6. List products that qualify for at least 10% discount.

```
SELECT p.ProdName, p.Price, d.DiscPct
FROM Product p
JOIN DiscountSlab d
ON p.Price BETWEEN d.MinPrice AND d.MaxPrice
WHERE d.DiscPct >= 10;
```

Steps:

1. Same non-equi join as Q5.
2. WHERE d.DiscPct >= 10 removes rows with 5% discount.
3. Remaining products: Notebook (10%), Bag (15%).